

Práctica 6. Maquetas tiflológicas.

Se trata de hacer toda la electrónica necesaria con un sensor de algún tipo (CNY70) de forma que, al pasar la mano a pocos milímetros active información en formato audible que se mantenga hasta que se vuelva a pasar la mano o que acabe la pista de audio. Usaremos estos elementos:

- Módulo de Lector de Tarjetas Micro SD TF de 6 Pines SPI con conversión de Nivel de Chip para Arduino UNO R3 Mega 2560 Due.
<https://es.aliexpress.com/item/32816126771.html>
- Micro SD Card 4GB. <https://es.aliexpress.com/item/1005003622701419.html>
- Módulo amplificador de Audio MAX98357 I2S.
<https://es.aliexpress.com/item/4001191391354.html>
- Altavoz de 8 Ohmios. <https://es.aliexpress.com/item/4000155879581.html>
- Sensor CNY70

Consumo de potencia:

Con audio apagado tenemos $5V * 7.5mA = 37.5mW$

Si batería total= 3000mAh, tendremos $3000mAh / 7.5mA = 400$ horas

Esto suponiendo que estará siempre sin funcionar.

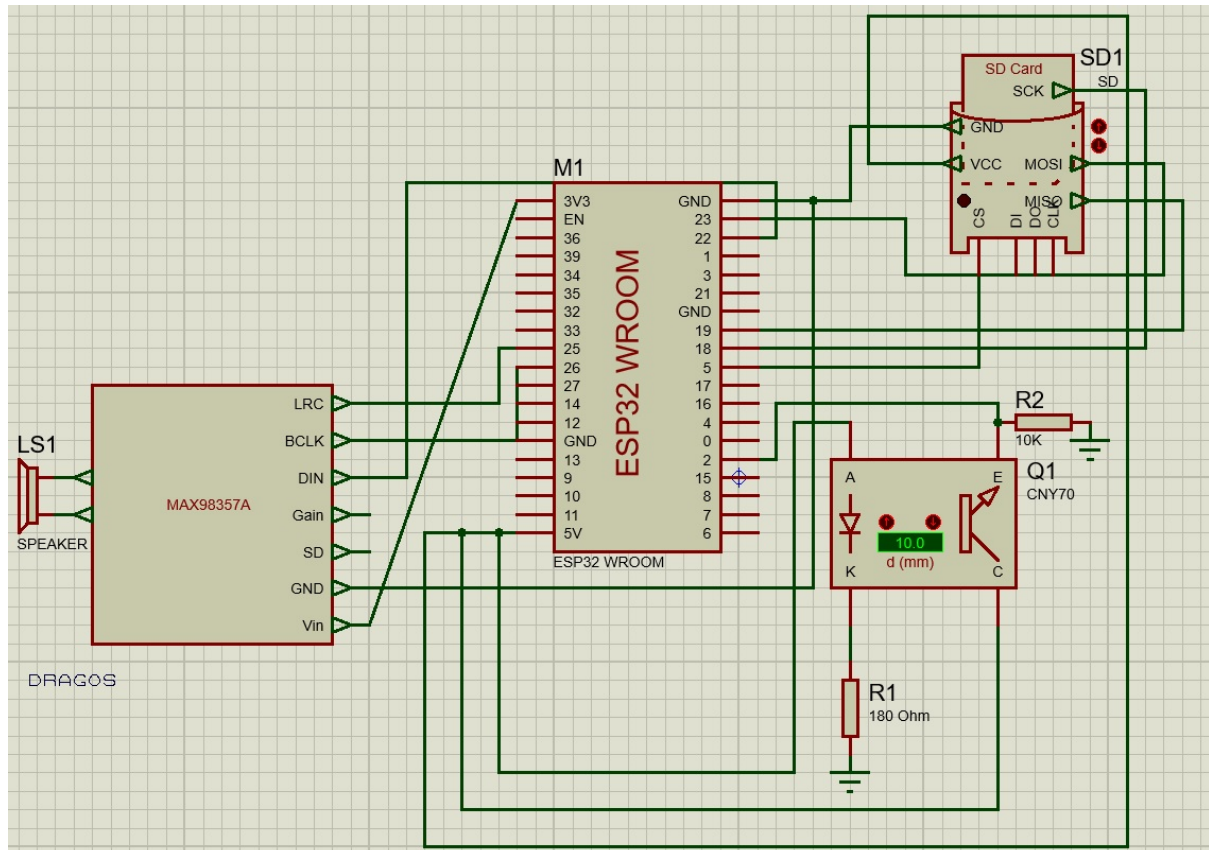
Con audio encendido tenemos $5V * 9.7mA = 48.5mW$

Si batería total= 3000mAh, tendremos $3000mAh / 9.7mA = 309$ horas aprox.

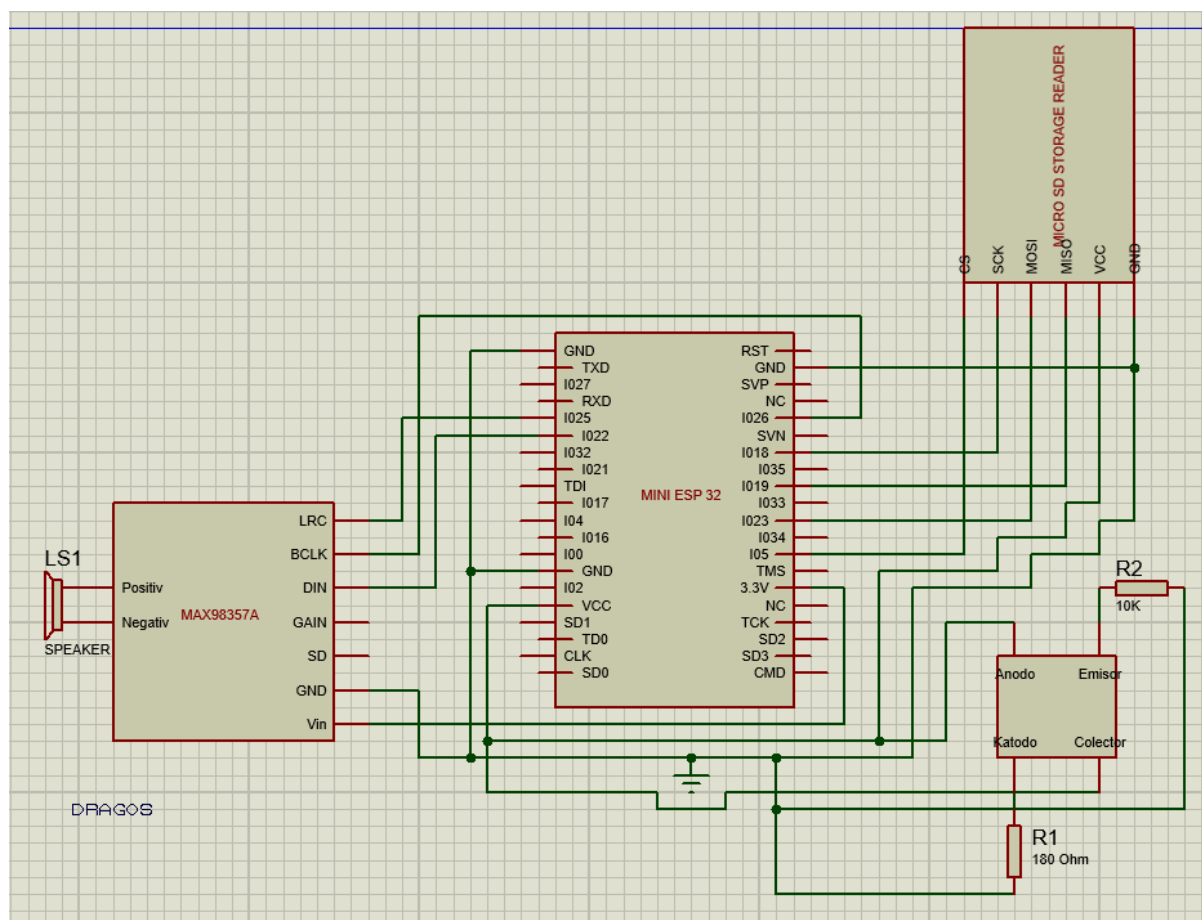
Esto suponiendo que está siempre funcionando.

Si funcionara de media 50%, estaría disponible durante 352 horas.

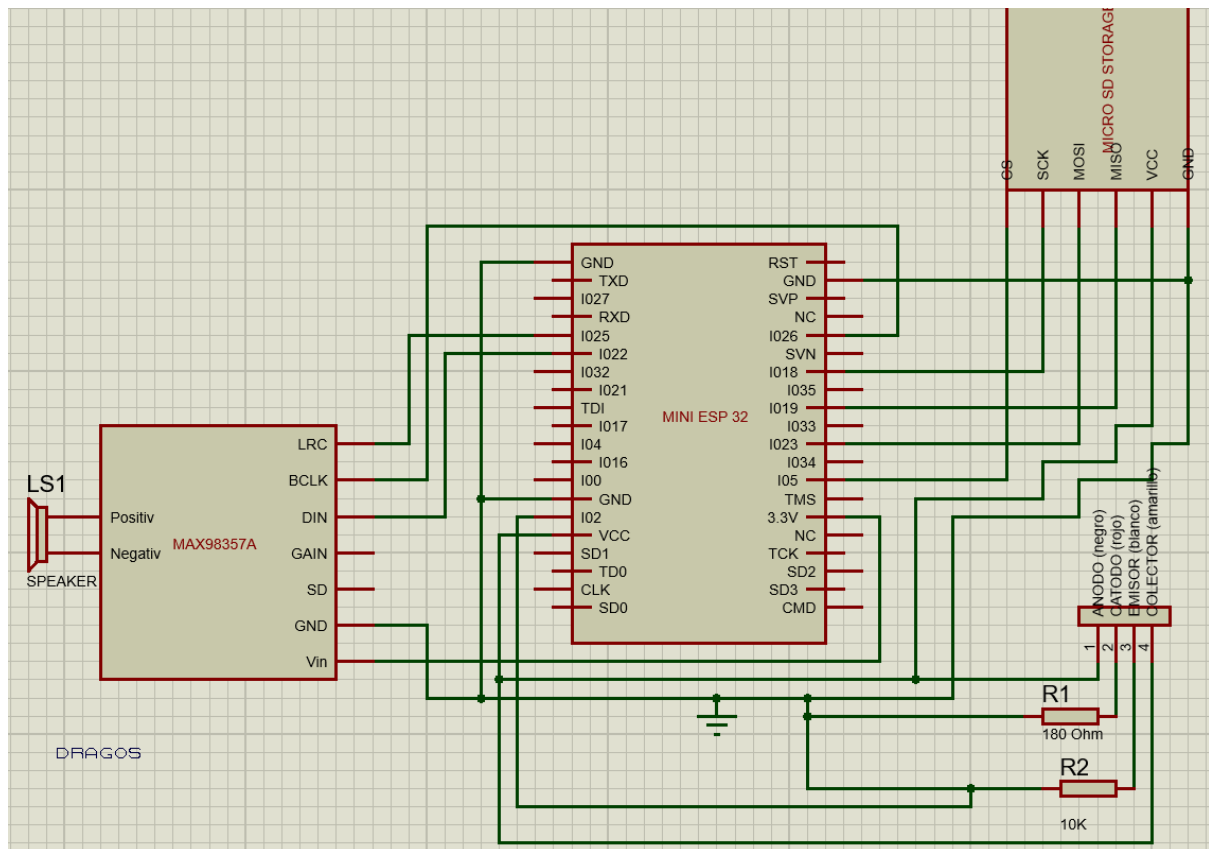
Esquema eléctrico.



Se han hecho varias modificaciones a continuación, debido a que los componentes no estaban creados y no hubiera sido posible trabajar sin estos en la PCB. Además hicimos otro cambio de microprocesador: hemos usado una mini Esp 32 en lugar de la ESP32 convencional, para optimizar el espacio. Ello ha contribuido a necesitar volver a diseñar el esquema:



Y volver a diseñar el esquema una vez más:



A continuación se adjunta el código que hemos usado para el funcionamiento de la maqueta tifológica. Al llevar un sensor CNY 70, este activa, desactiva y reinicia el sonido preparado y guardado en la micro SD, tal y como se ha descrito en la práctica correspondiente de la maqueta tifológica del trimestre pasado.

```
#include <Audio.h>
#include "SD.h"
#include "FS.h"

//Digital I/O used
#define SD_CS 5
#define SPI_MOSI 23 // SD card
#define SPI_MISO 19
#define SPI_SCK 18

#define I2S_DOUT 25
#define I2S_BCLK 27 //I2S
#define I2S_LRC 26

Audio audio;
```



```

//VARIABLES GLOBALES

boolean musicaON = false;

//ENTRADAS
#define PIN_SENSOR 2

unsigned int valorSensor = 0; //unsigned int (enteros sin signo) son lo
mismo que los int que almacenan un valor de 2 bytes. En lugar de
almacenar números negativos, solo almacenan valores positivos,
produciendo un rango de 0 a 65,535 (2^16) - 1).
unsigned int UMBRAL = 110;

void lecturaSensor()
{
    valorSensor = analogRead(PIN_SENSOR);

    if (valorSensor >= UMBRAL ) //condición, si el valor que devuelve el
sensor es mayor o igual que el umbral establecido, tendremos que en base
a ello el audio se activa o se pausa
    {
        musicaON = !musicaON;
        Serial.println("musicaON1");

        delay(500);
        audio.connecttoFS(SD, "/pikusus.mp3");
    }
}

void accionesSalida()
{
    if (musicaON) {
        Serial.println("musicaON2");
        audio.loop();
    }
}

void setup() {
    pinMode(PIN_SENSOR, INPUT); // sensor como entrada

    pinMode(SD_CS, OUTPUT); //establecemos tarjeta SD como salida
porque manda info para que se lea
}

```

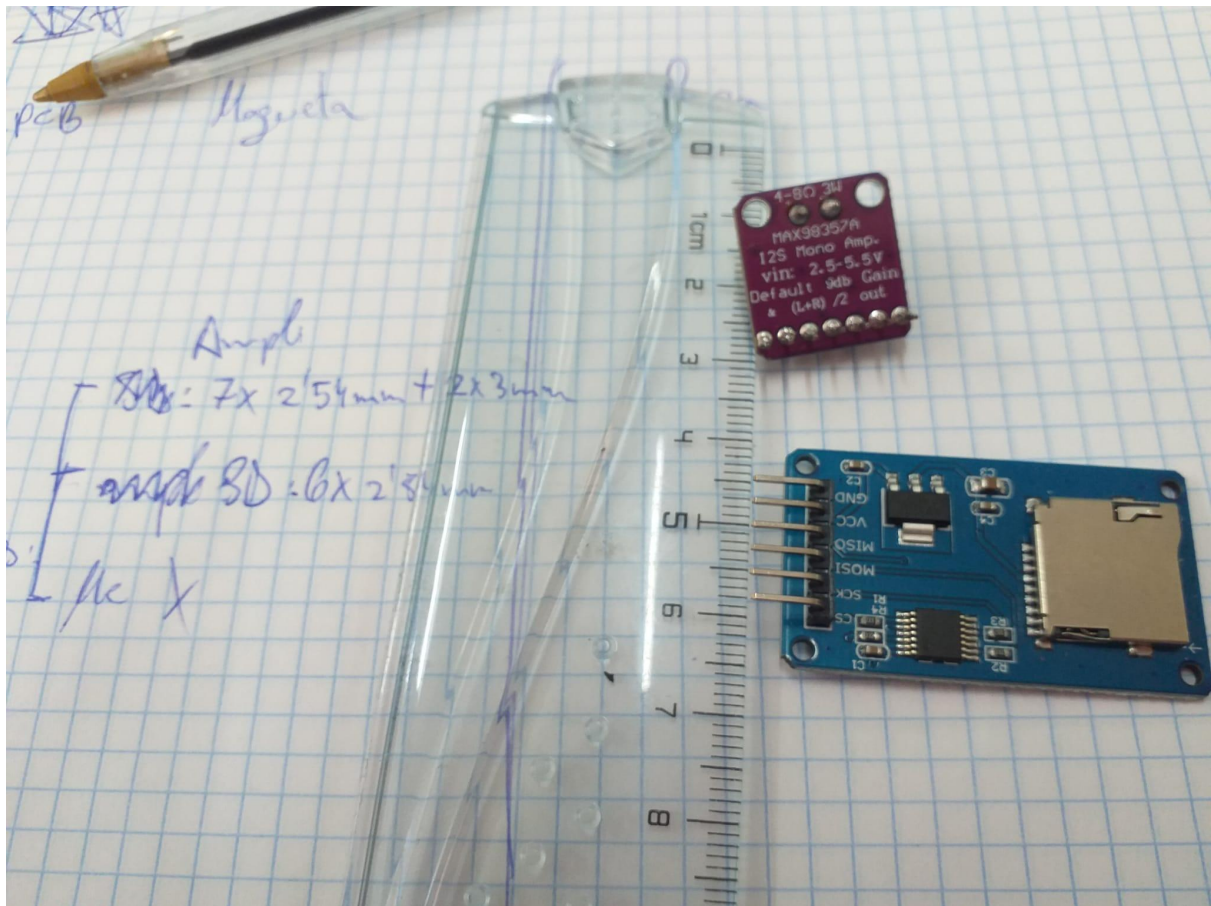
```

digitalWrite(SD_CS, HIGH);
SPI.begin(SPI_SCK, SPI_MISO, SPI_MOSI);    //empezamos la comunicación
con la SPI
Serial.begin(115200);
if (!SD.begin(SD_CS))                    //establecemos que si no es posible por
Software la comunicación con la tarjeta SD, nos saldrá el mensaje de
"Error..." en el monitor. Nos permite identificar errores rápidamente al
segmentar la práctica en varias partes
{
    Serial.println("Error talking to SD card!");
    while (true); //establecemos que se mantenga en bucle
while mientras persista el error
}
audio.setPinout(I2S_BCLK, I2S_LRC, I2S_DOUT);    //establecemos
relación entre amplificador y speaker como salida
audio.setVolume(21); //ajuste volumen de 0...21
audio.connecttoFS(SD, "/pikasus.mp3");    //para que se lea el
archivo de la tarjeta SD. Tiene que ponerse aquí el mismo nombre de
archivo que hay en el SD, y no puede superar 8 letras
}

void loop() {
    lecturaSensor();
    Serial.println(valorSensor);
    accionesSalida();
}

```

Hemos tenido que crear la huella de todos los componentes al no existir en Proteus. En primer lugar hemos medido los componentes uno por uno, para crear su versión virtual.



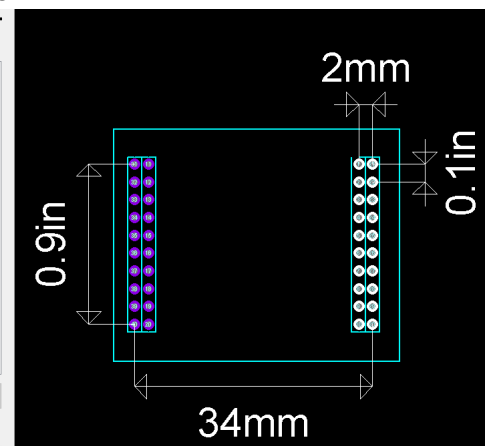
Luego hemos seleccionando el elemento en la parte del ISIS de Proteus, click derecho, y seleccionamos "Packaging tool". Luego seleccionamos cada pin y establecemos el número y la descripción del pin. En la parte superior nombramos nuestro elemento, en este caso "Mini ESP32". Por último seleccionamos "Assign Packages" en la parte inferior derecha.

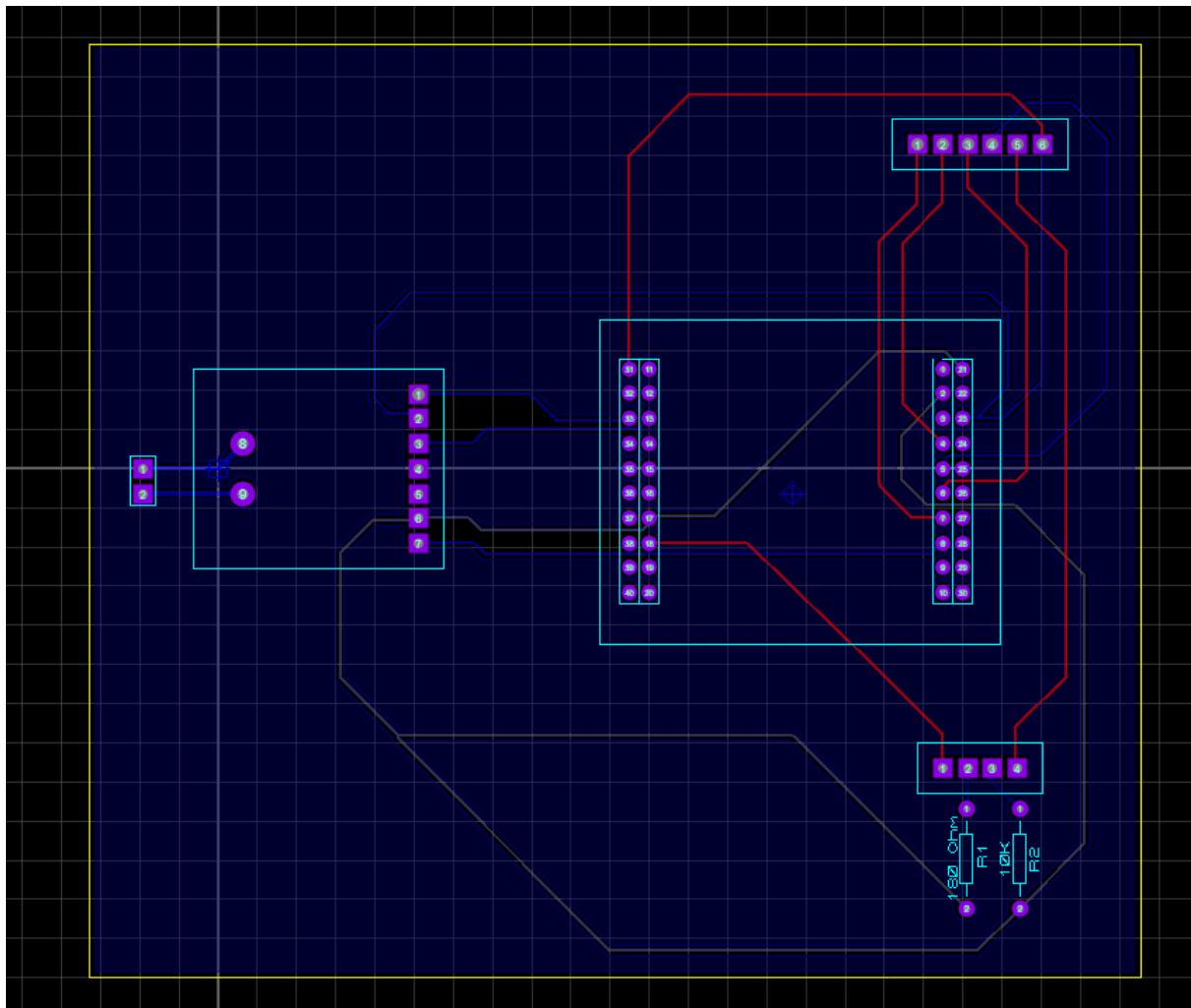
No. Of Gates: 1 ☐ Gates (elements) can be swapped on the PCB layout?

Pin	Hidden	Common	Type	A
3.3V			Passive	8
CLK			Passive	
CMD			Passive	30
GND[00]			Passive	21
GND[01]			Passive	
GND[02]			Passive	
I00			Passive	
I02			Passive	
I04			Passive	
I05			Passive	7
I016			Passive	
I017			Passive	
I018			Passive	4
I019			Passive	5
I021			Passive	
I022			Passive	
I023			Passive	6
I025			Passive	
I026			Passive	

NC Pins: Add Pin Remove Pin

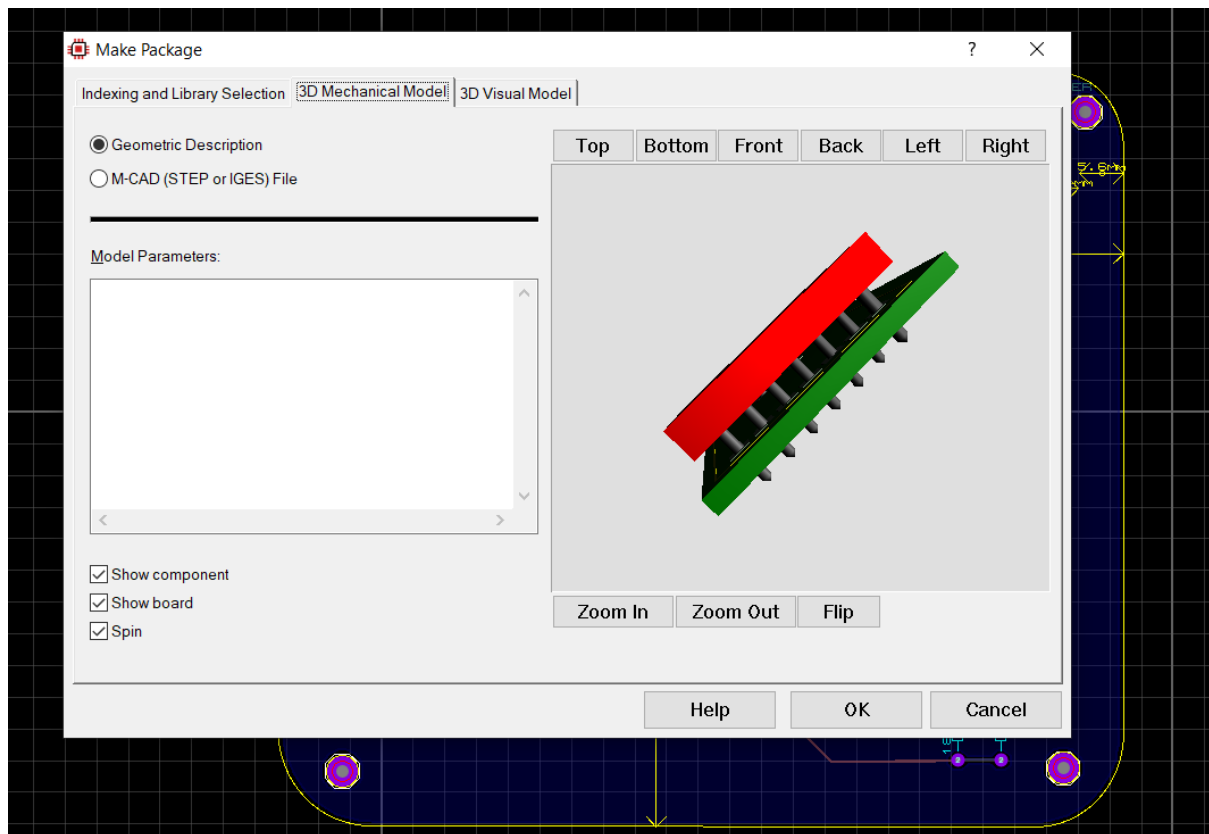
Swappable Pins:



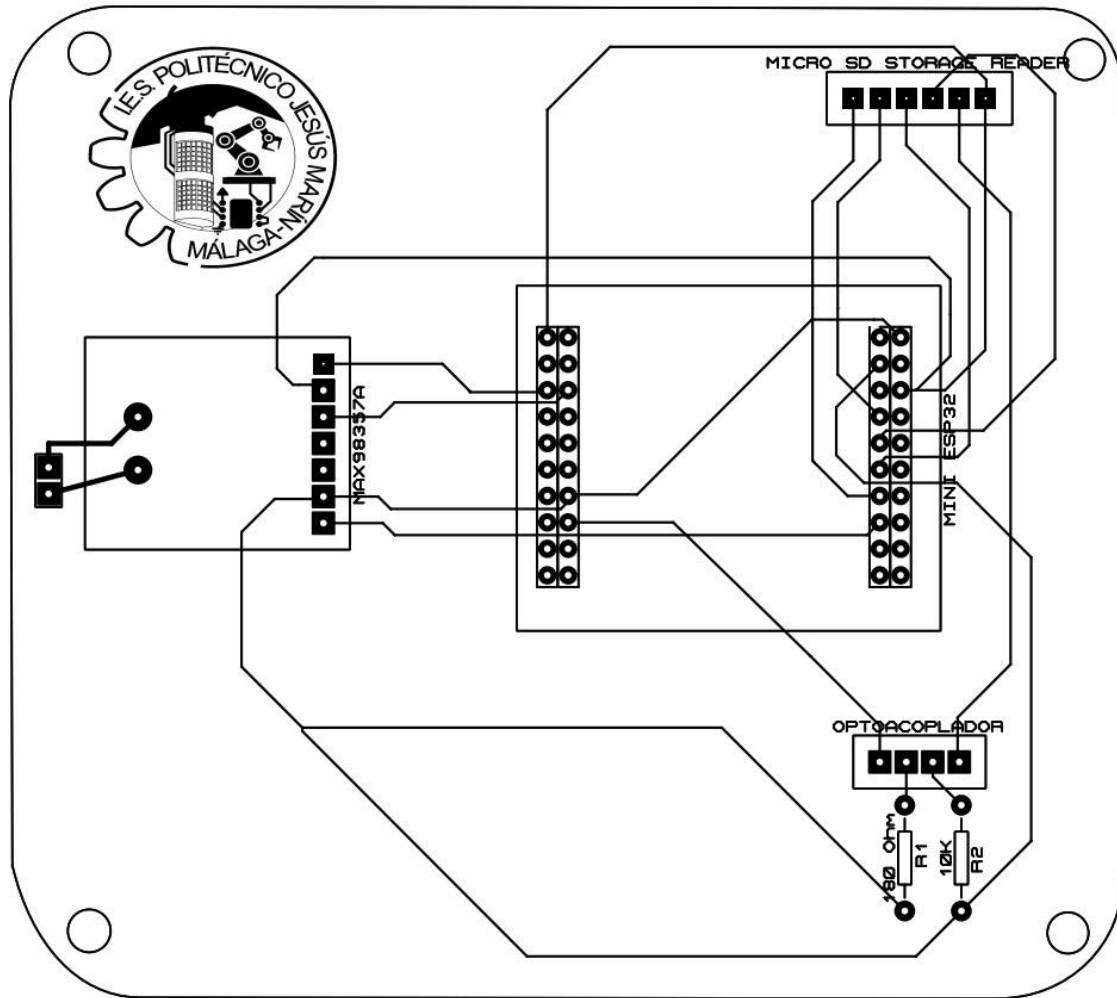


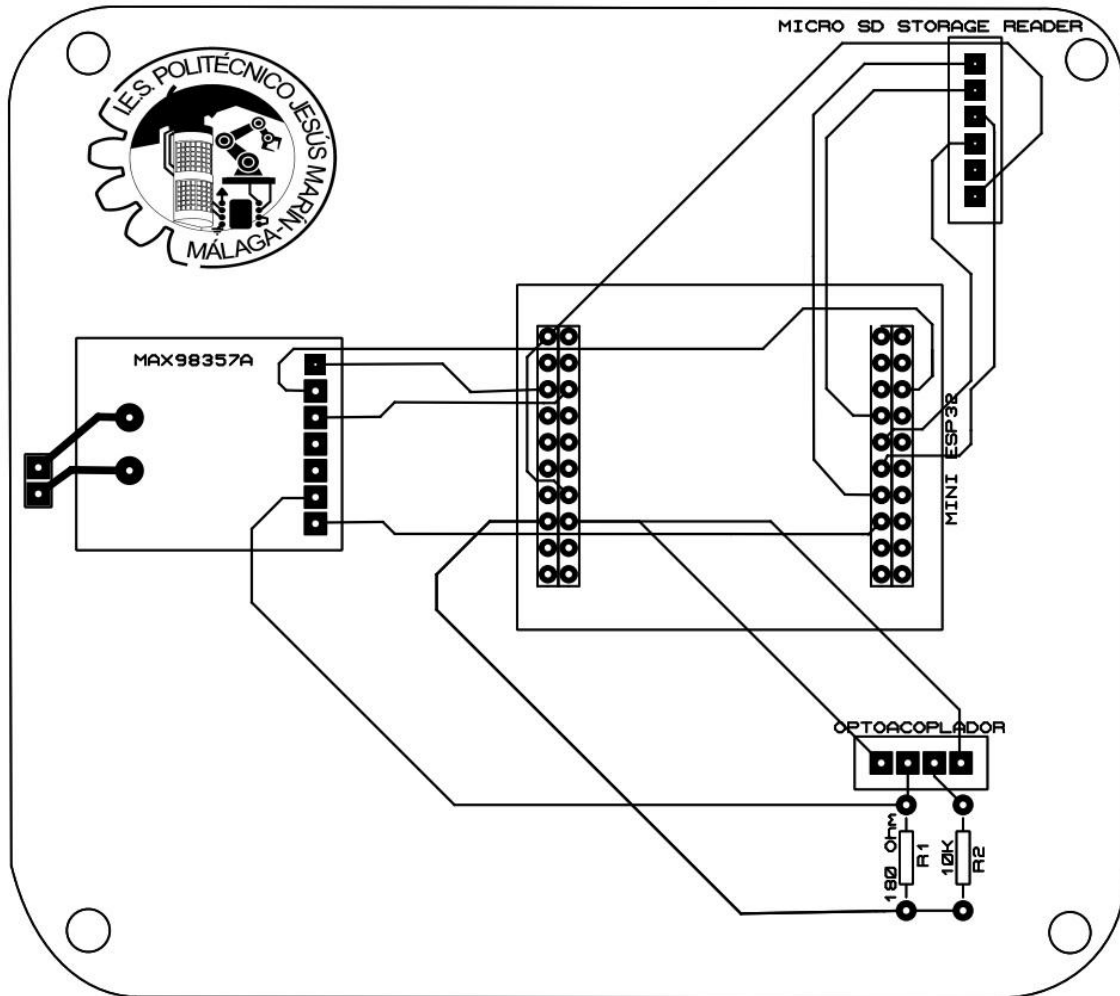
En la parte del ARES de proteus, hemos ido creando los elementos uno por uno, y el conexionado, ya que la aplicación no nos ha dejado aplicar ni el autoruteo ni el auto-placement.

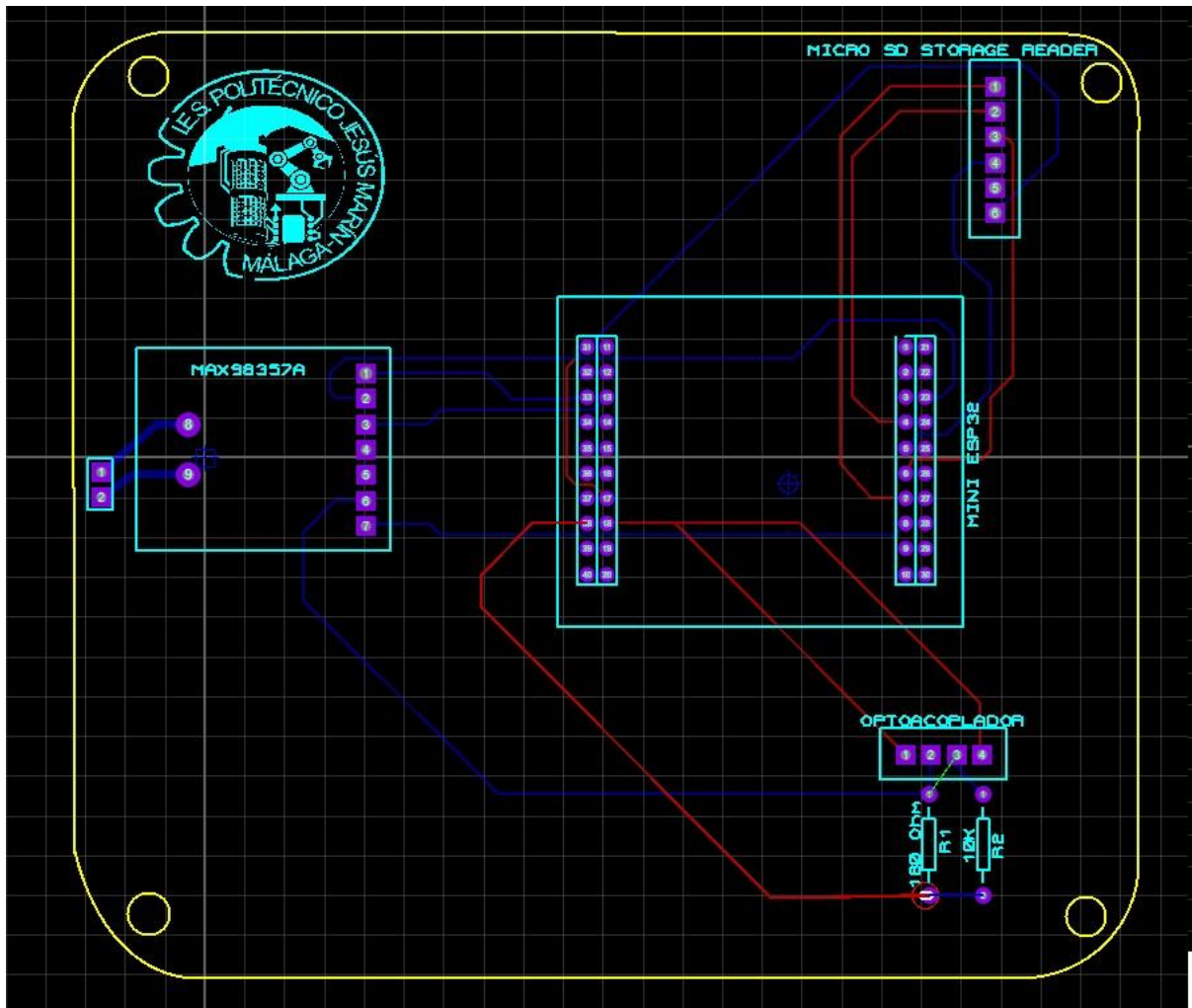
Para crear el elemento, hemos pinchado en este, click derecho, “Make Package”, en la categoría del circuito hemos seleccionado “Integrated Circuits”. Al seleccionar Package Type, hemos elegido Through Hole. Y en “Package description” hemos escrito “Zócalo para conectar max98357a”. Proteus nos permite también ver una imagen 3D del componente:



Para uno de los últimos detalles, se nos había olvidado redondear la placa, añadir el logotipo del centro y marcar los huecos para el posterior taladrado, según nos había pedido el cliente. Esto se rectificó rápidamente con ayuda de un par de tutoriales de youtube.



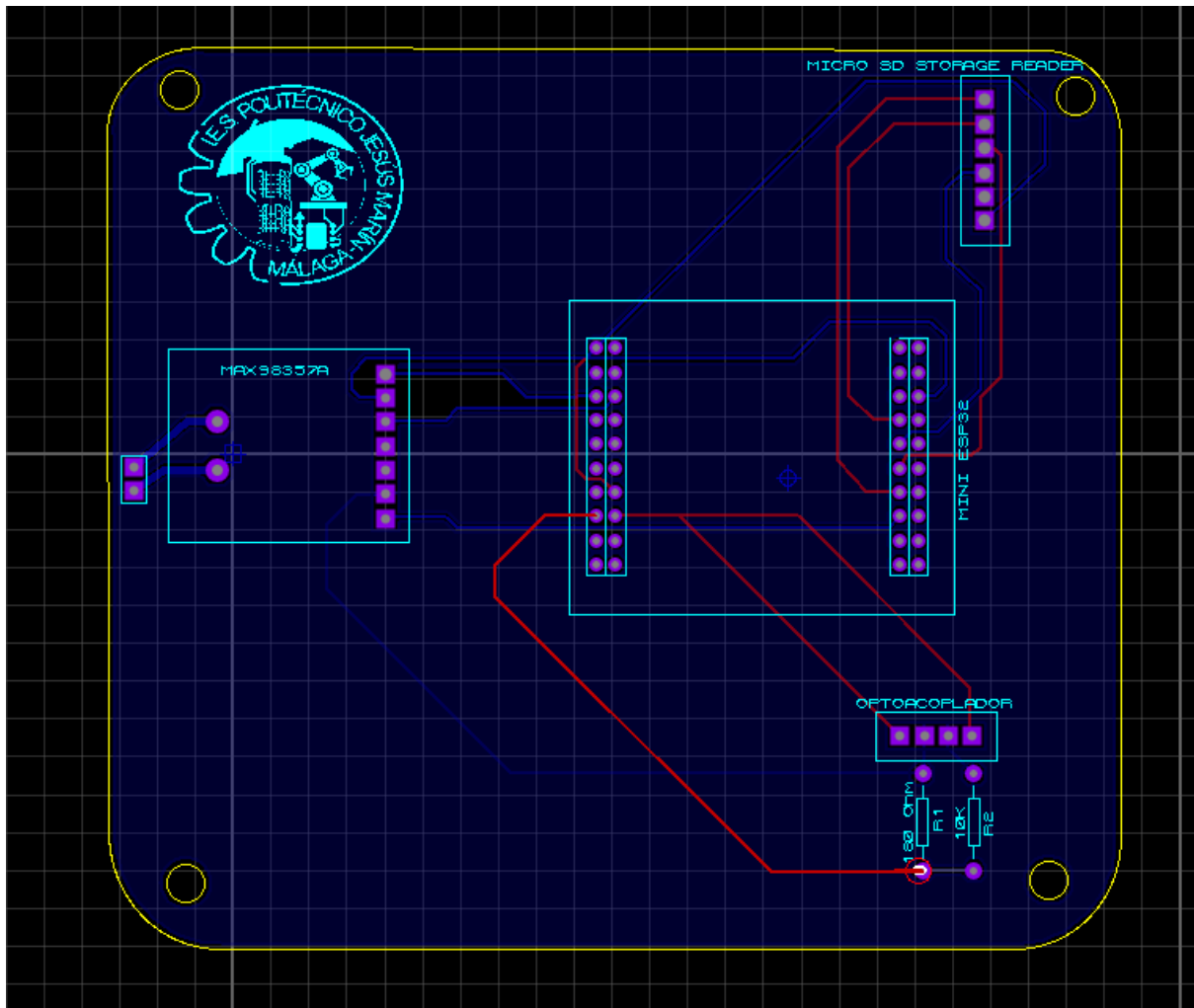




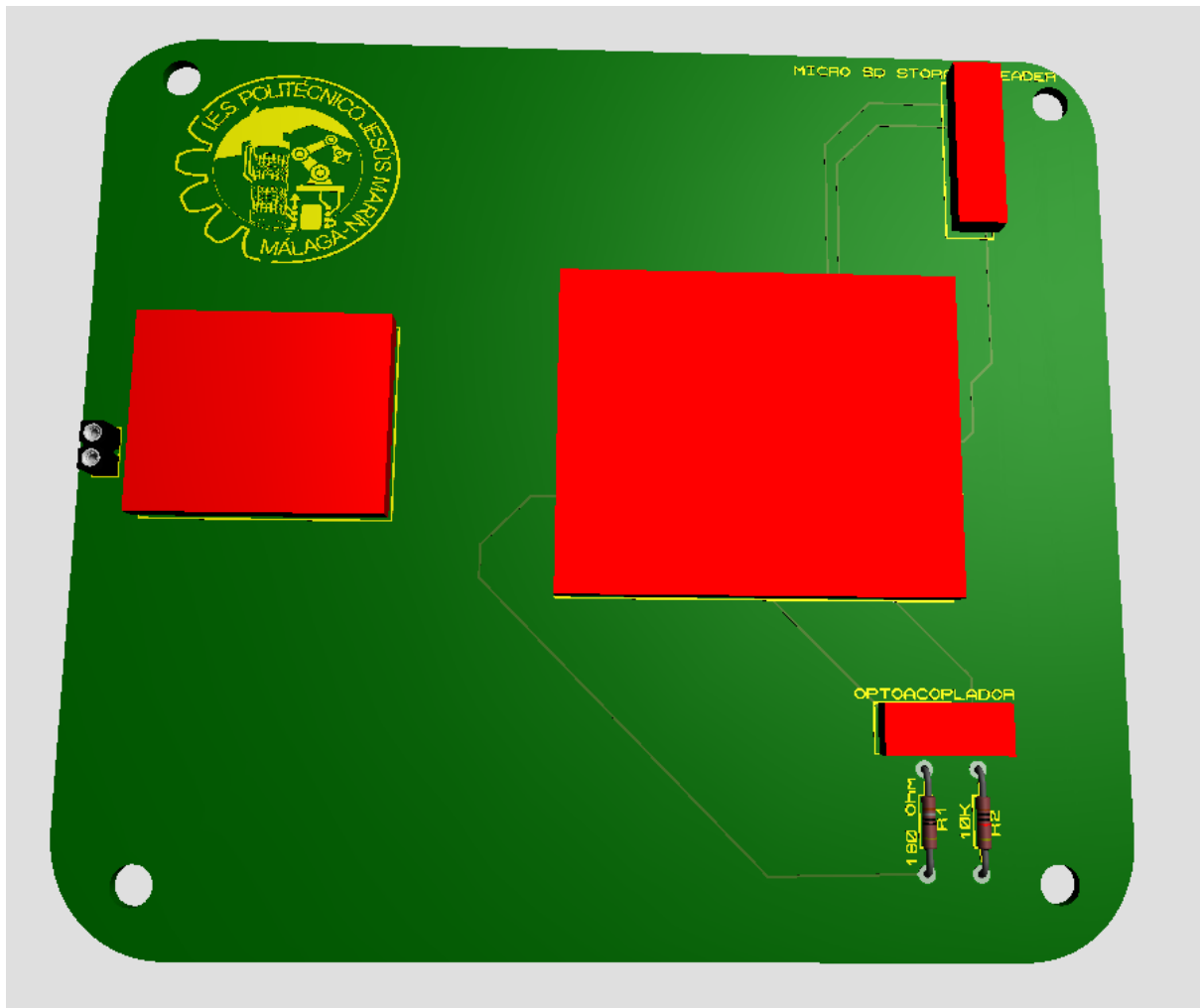
Más errores que nos hemos encontrado:

Se ha arrastrado el mismo problema desde el principio, que no hemos sabido resolver. Y es que al crearse componentes en la parte esquemática, estos no se traspasaron a la parte de la PCB. Hemos intentado crear la huella, y Proteus tampoco te reconocía los componentes. Esto se tradujo que hacia el final del diseño, no fue posible hacer un buen plano de tierra, y el diseño en FlatCam quedó con pistas incompletas, y sin la placa de fondo.

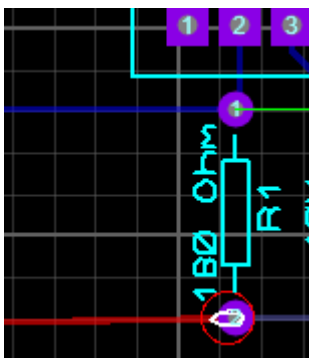
Esto se arregló rehaciendo el redondeo de la placa, con mucho más zoom y cuidado a la hora de unir los bordes de la placa. Ello permitió que el plano de tierra estuviera ya correcto, con estos resultados:



Lo que permitió también la visualización 3D:

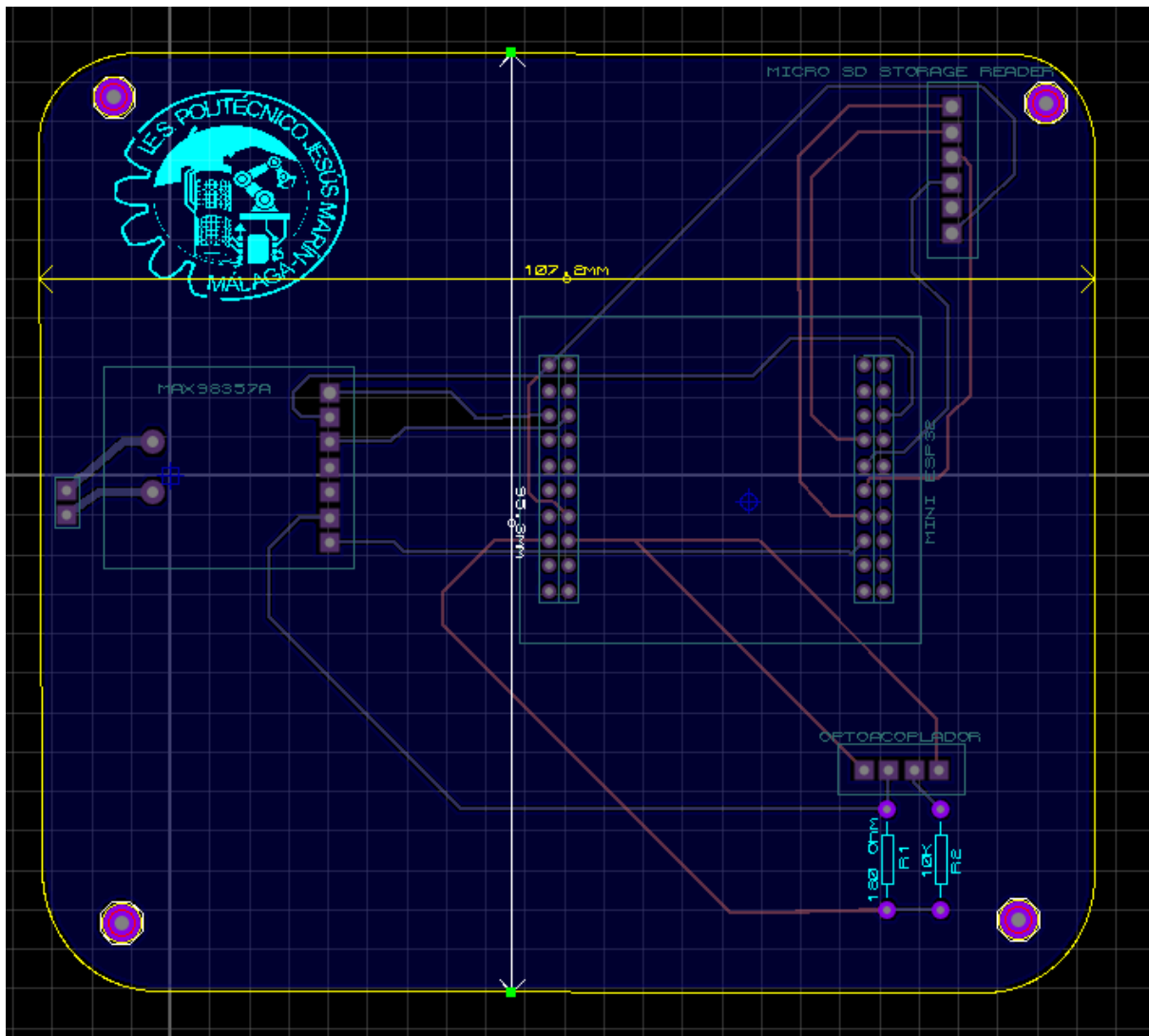


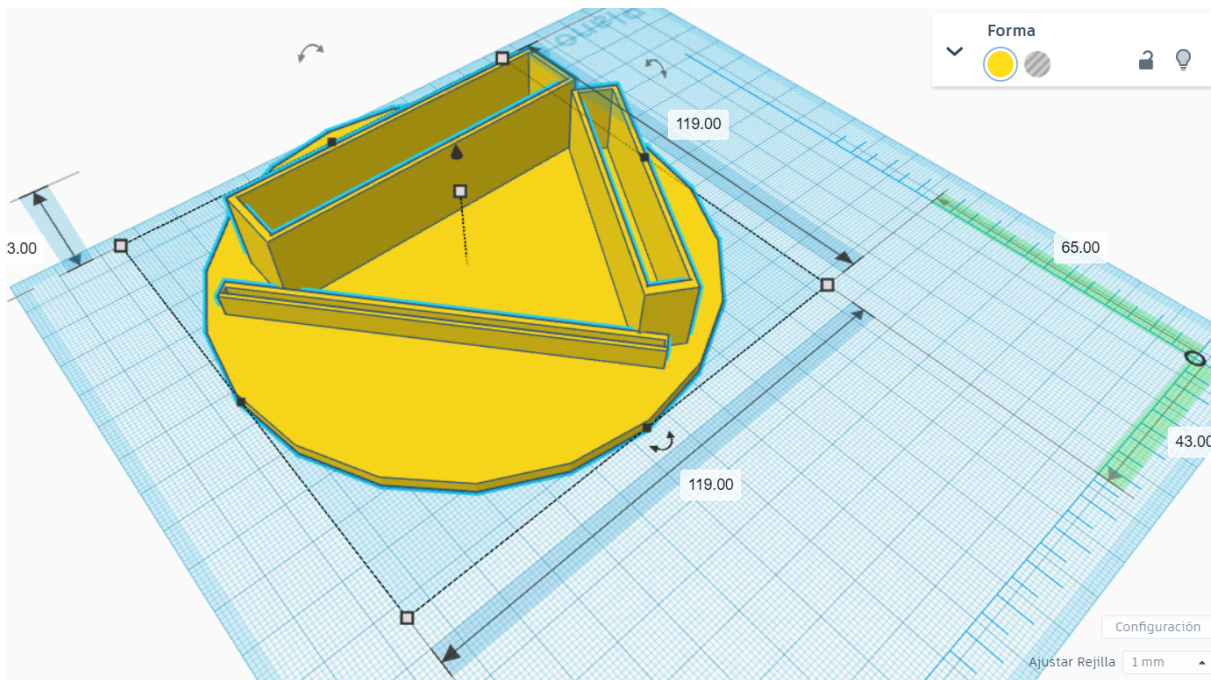
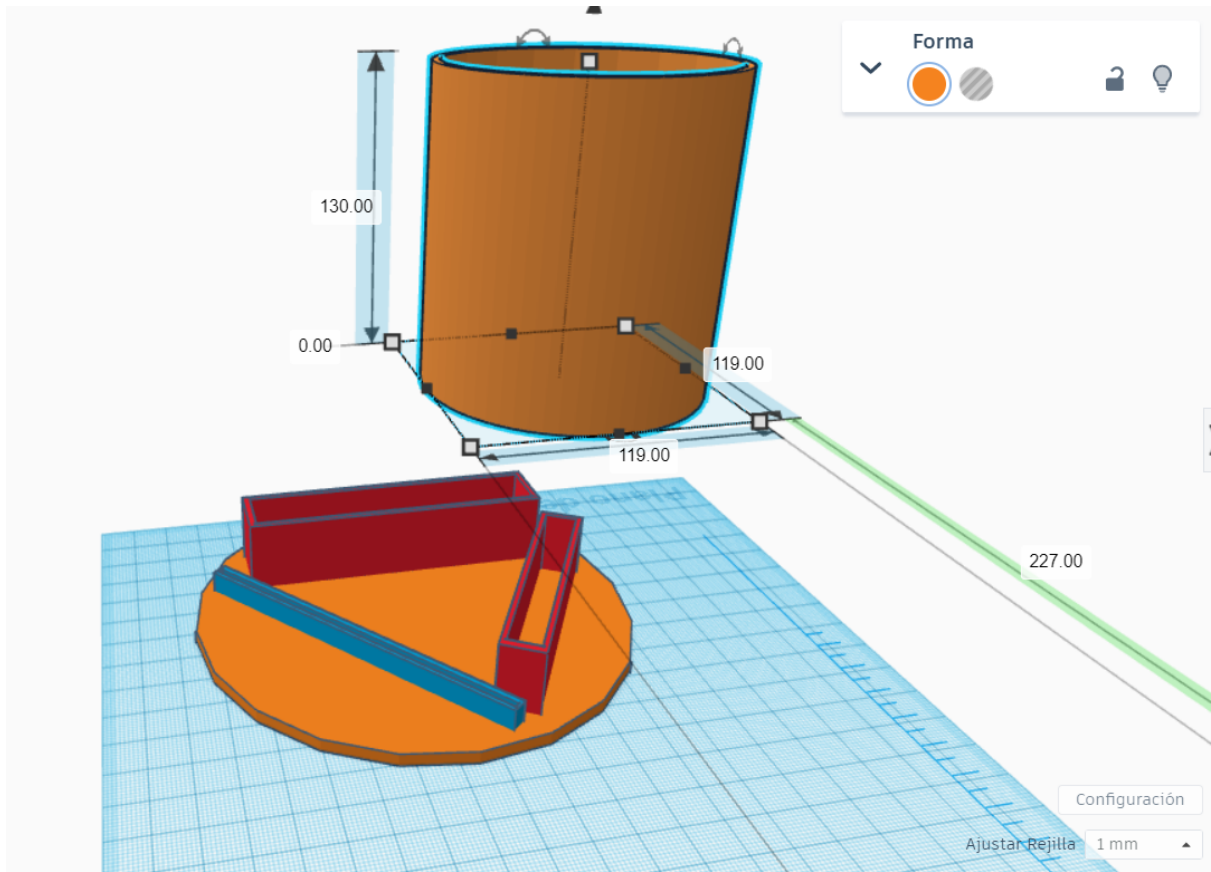
1. He intentado instalar la última versión de Proteus para intentar escribir la serigrafía:

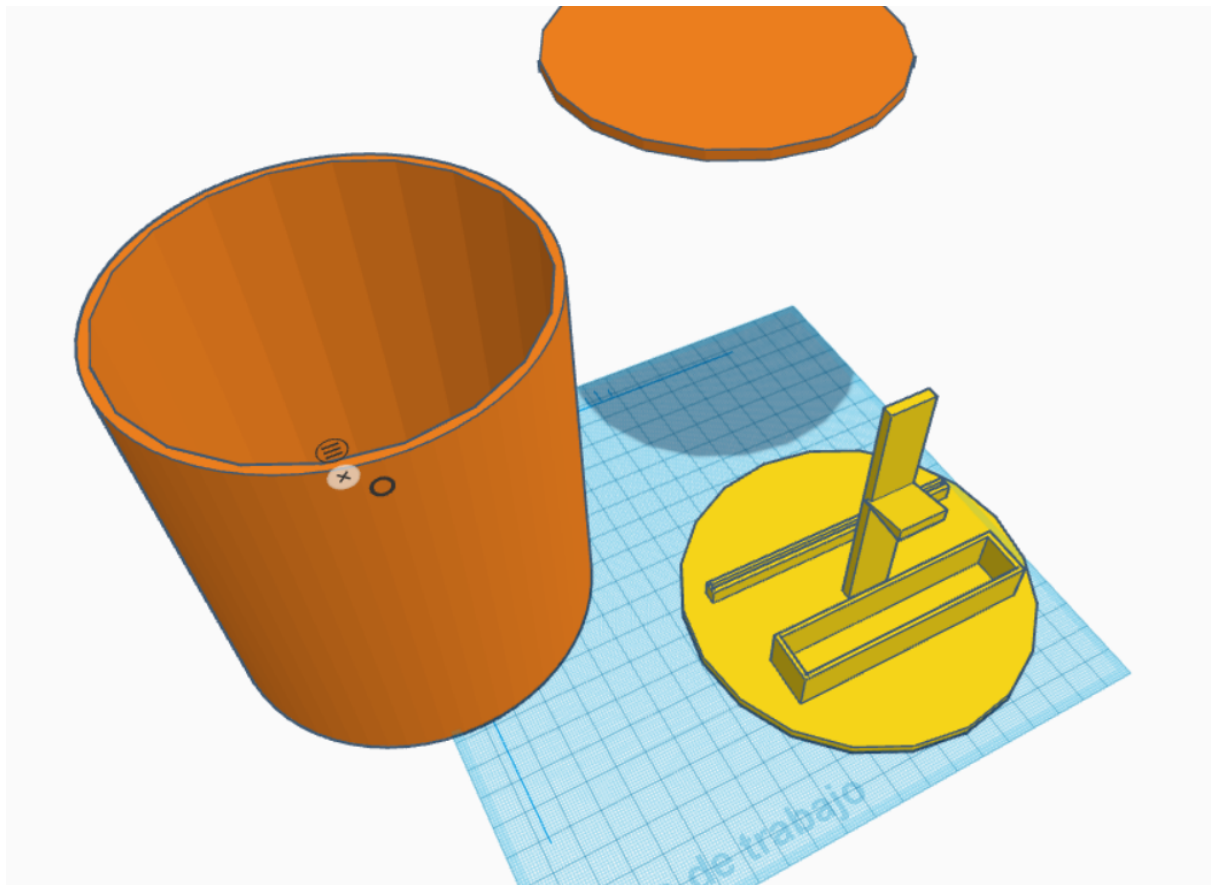


Sin embargo no lo he conseguido. La serigrafía adquiere un color “fantasmal” como de segundo plano, cuando se vuelve a abrir la aplicación.

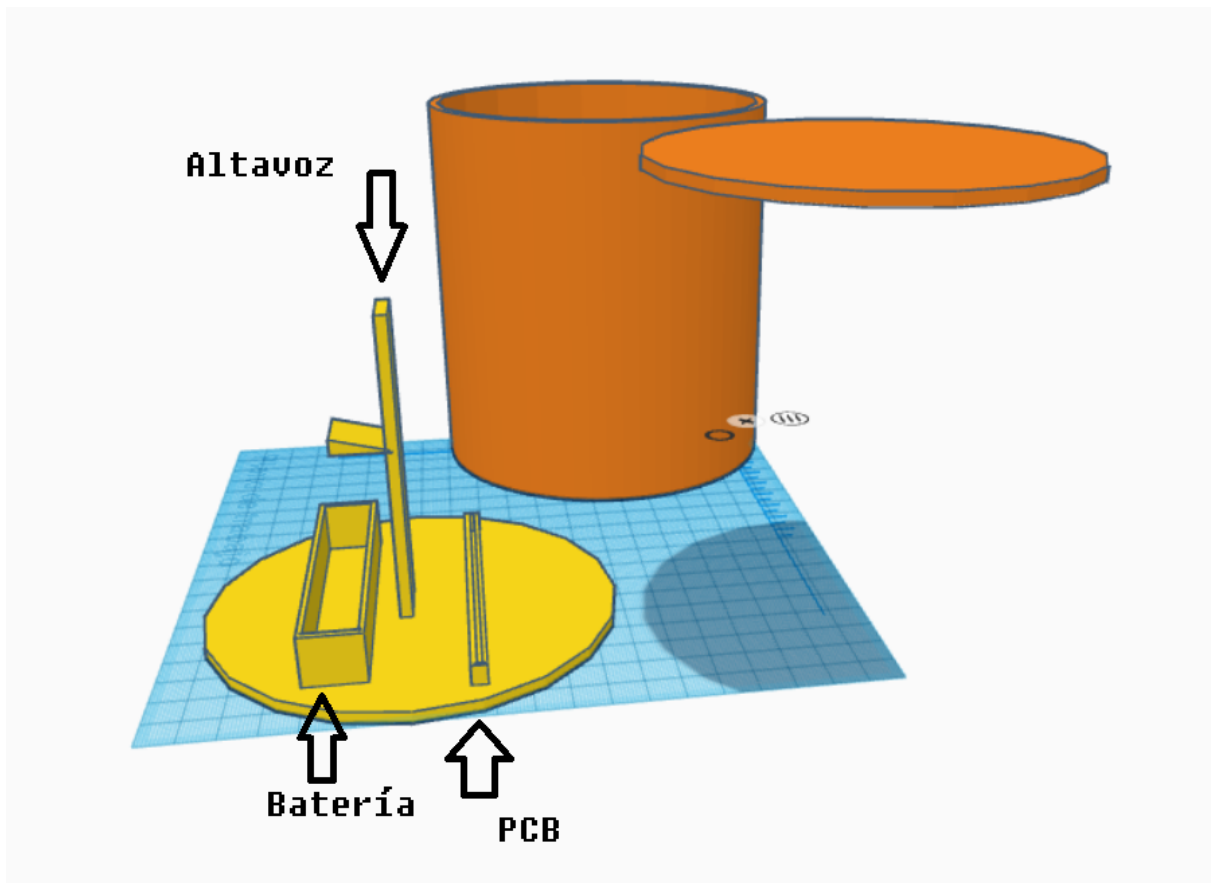
En el siguiente paso, hemos procedido a tomar mediciones para preparar la impresión 3D que va a contener nuestra placa, la batería, y el altavoz del conjunto de la maqueta tiflológica.

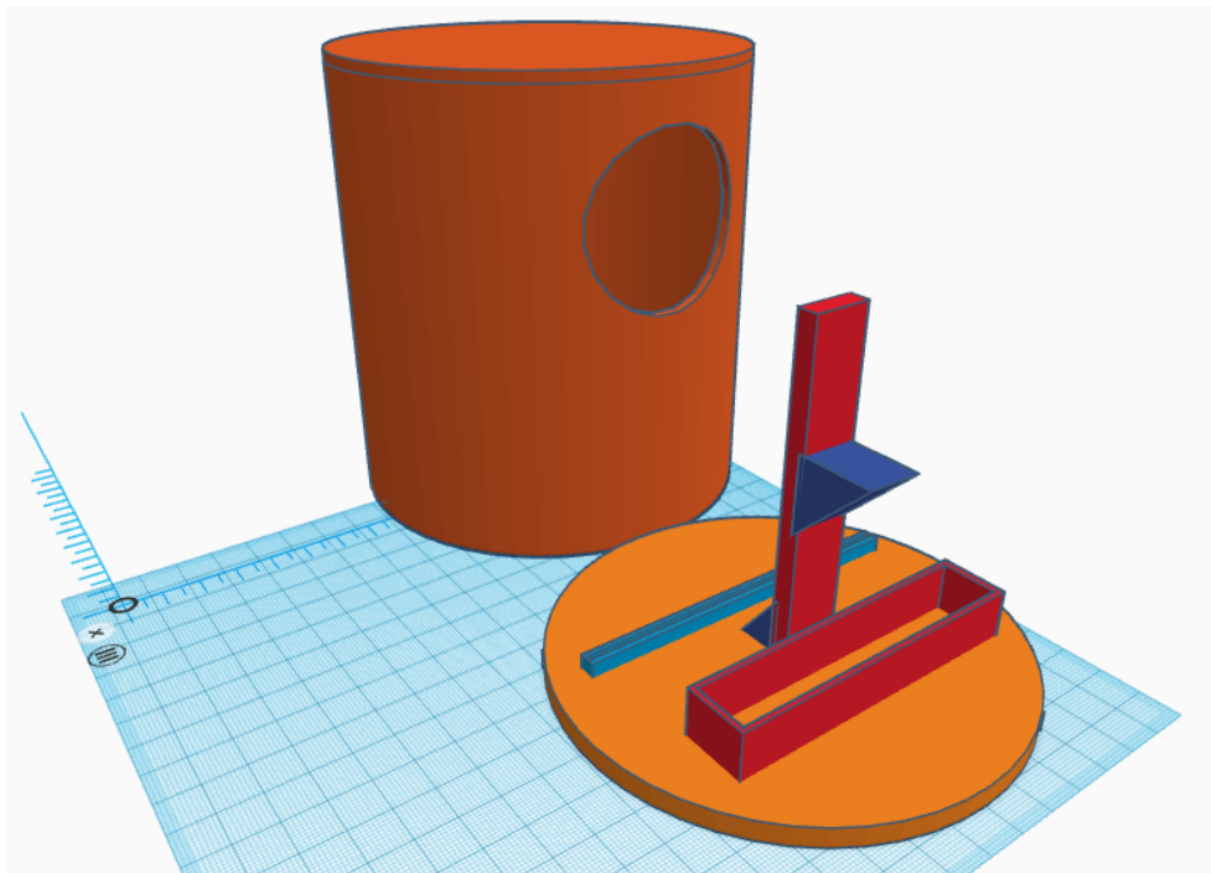
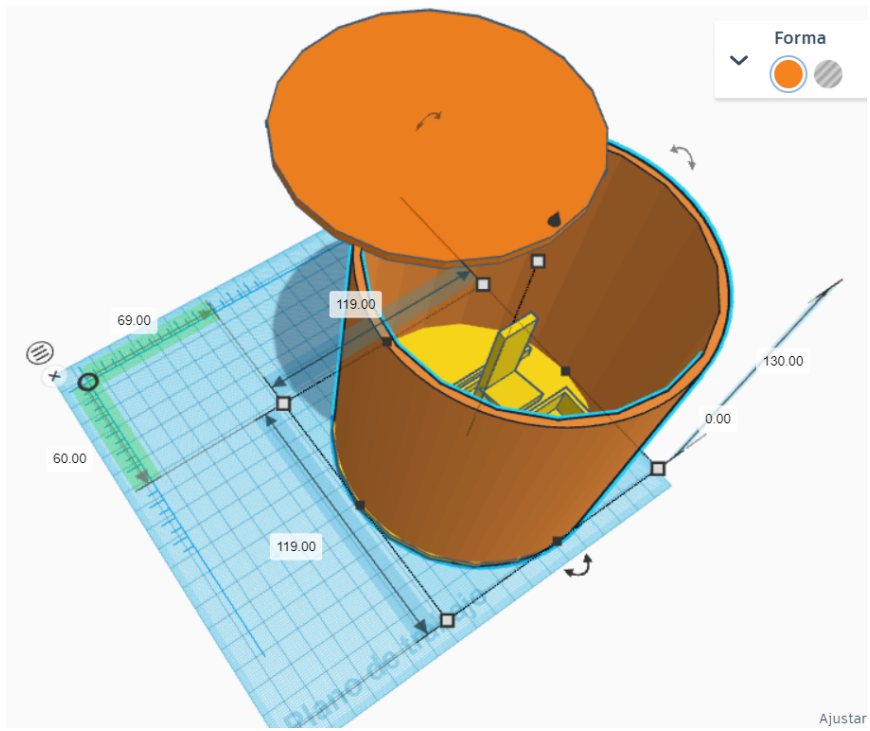






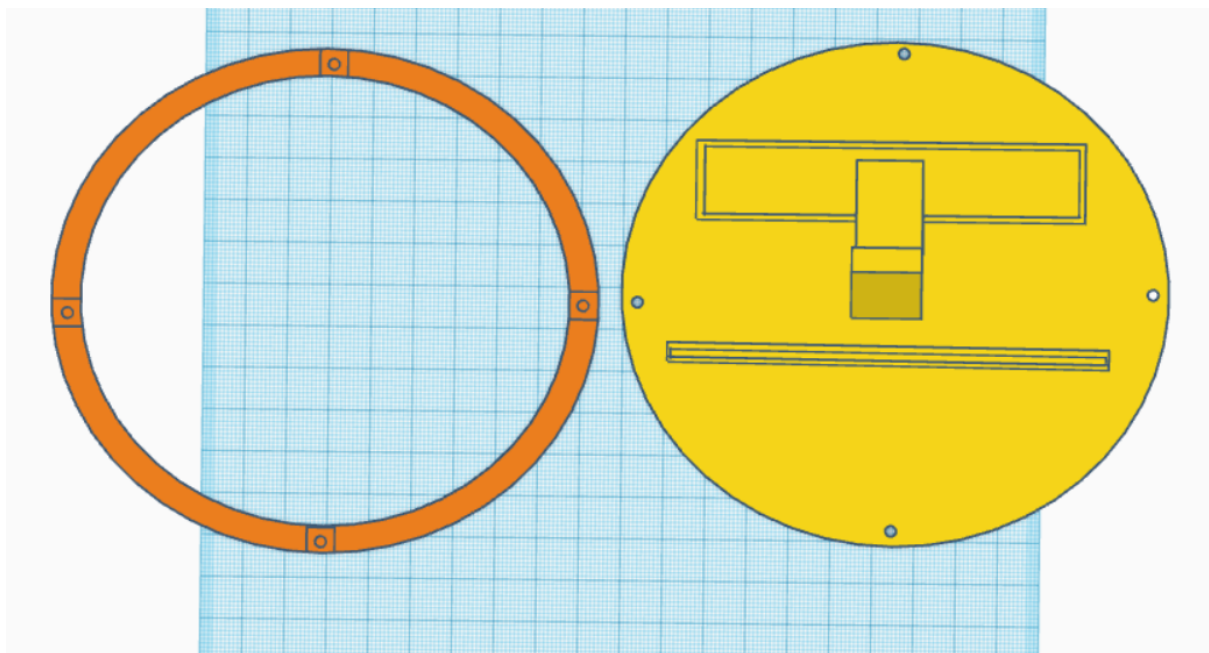
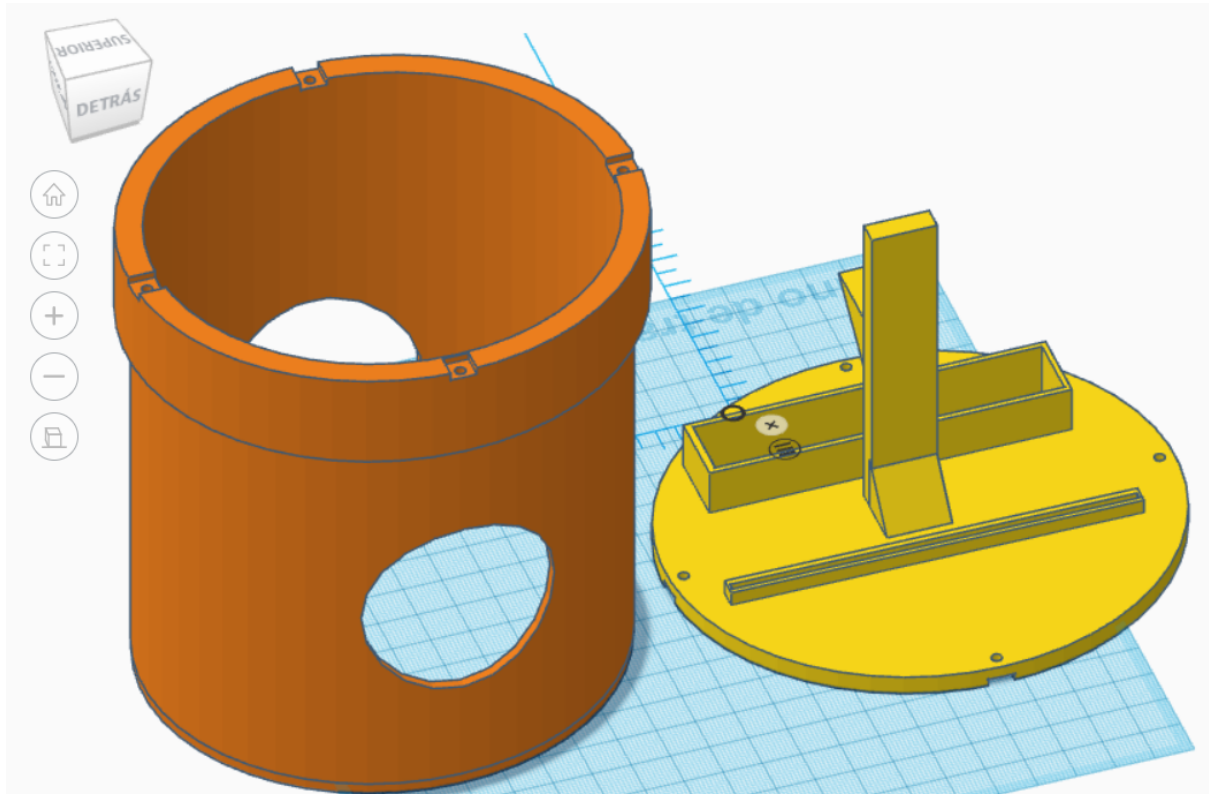
En estas imágenes se ven ya los segundos prototipos.







Para encajar las 2 piezas, hemos barajado varias opciones. Nos hemos quedado con la opción del atornillado. Para ello hemos necesitado un calibre para averiguar el tamaño apropiado para nuestro tornillo, teniendo en cuenta el grosor disponible de las dos piezas. Resulta que vamos a necesitar 4 tornillos, de 3mm diámetro cada uno, y las tuercas serán de 6 mm de diámetro.





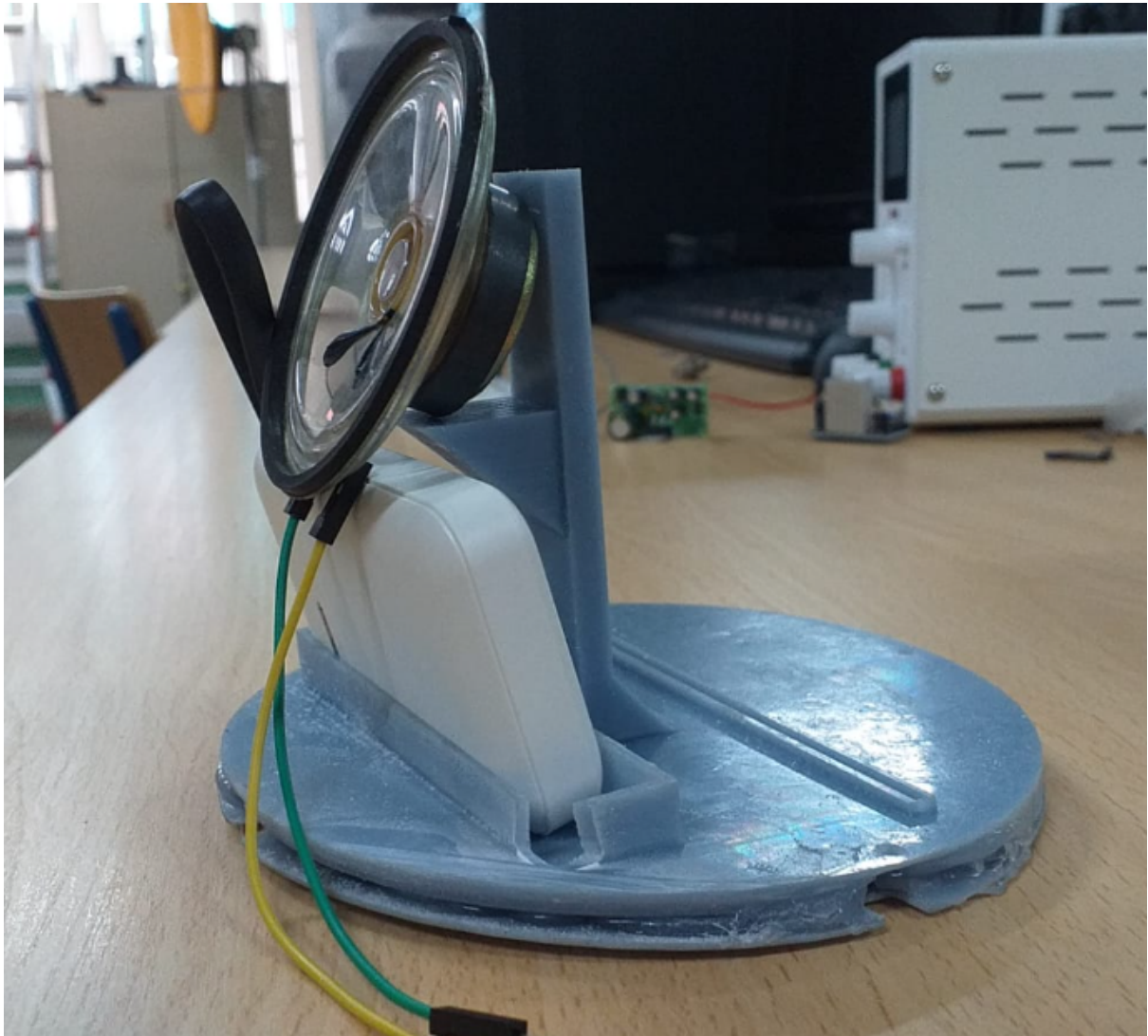
En esta imagen se muestra el fallo del primer intento de impresión 3D. Resulta que no nos dimos cuenta de que la tapa superior estaba hueca, y la impresora imprimió erróneamente el conjunto por no haberse podido articular las piezas correctamente.

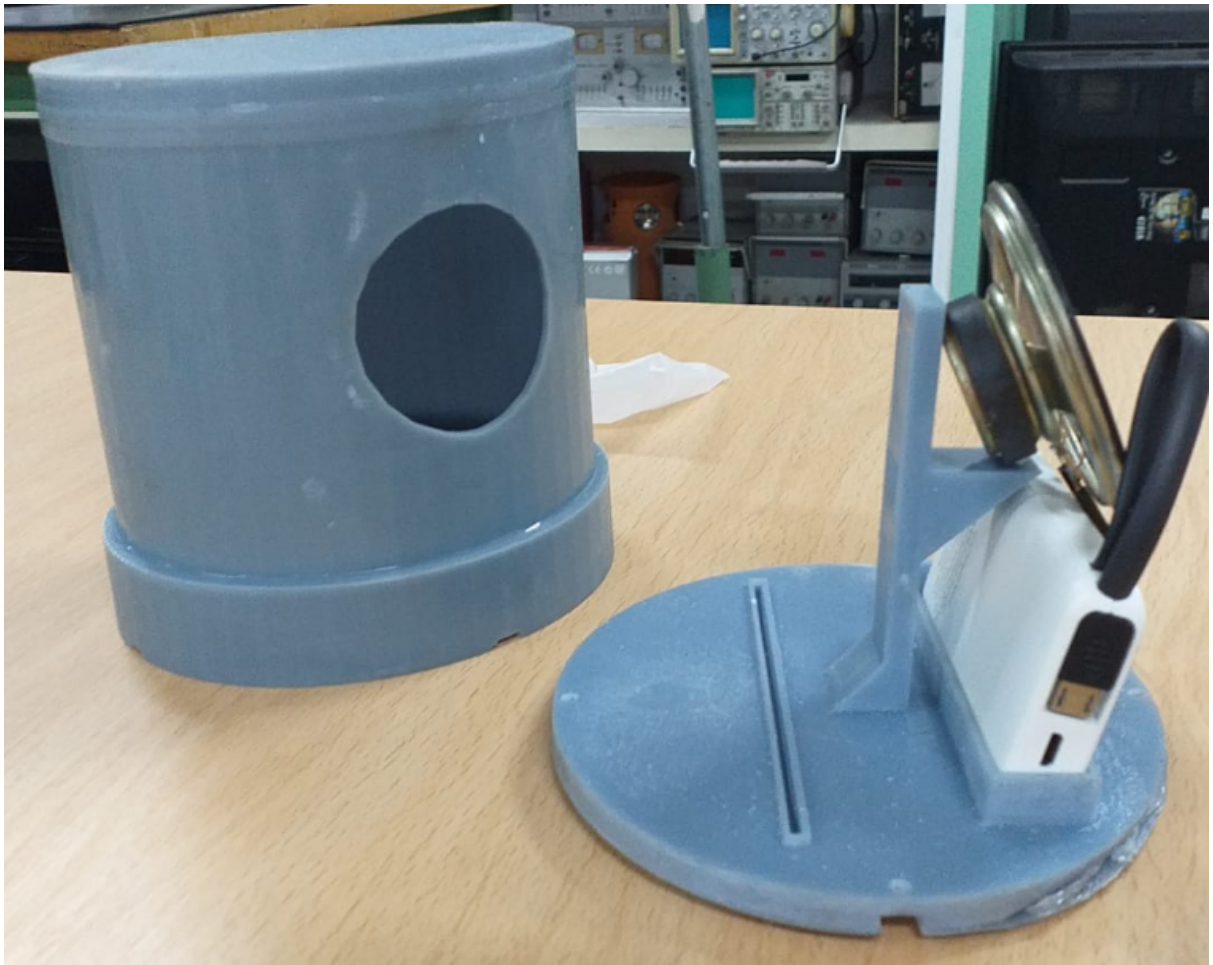
Diagnóstico de fallos de la impresora 3D:

Después de varios intentos, el profesor se ha encontrado con los siguientes “fallos” y se ha rectificado de la siguiente manera:

1. Filtrado de la resina
2. Limpiar impresora cada pieza que pasa por el láser, pantalla, etc.
3. Ajustar nivelado de la plataforma
4. Ajustar la potencia de la radiación ultravioleta. Estaba en 65%, y finalmente se puso al 90%

Los siguientes resultados fueron satisfactorios; salvo pequeños detalles, como los agujeros donde irían los tornillos. La impresión no ha respetado el diámetro de 3mm, y en algunos casos estaban llenos de resina. He tenido que usar el taladro para ahuecar las zonas.







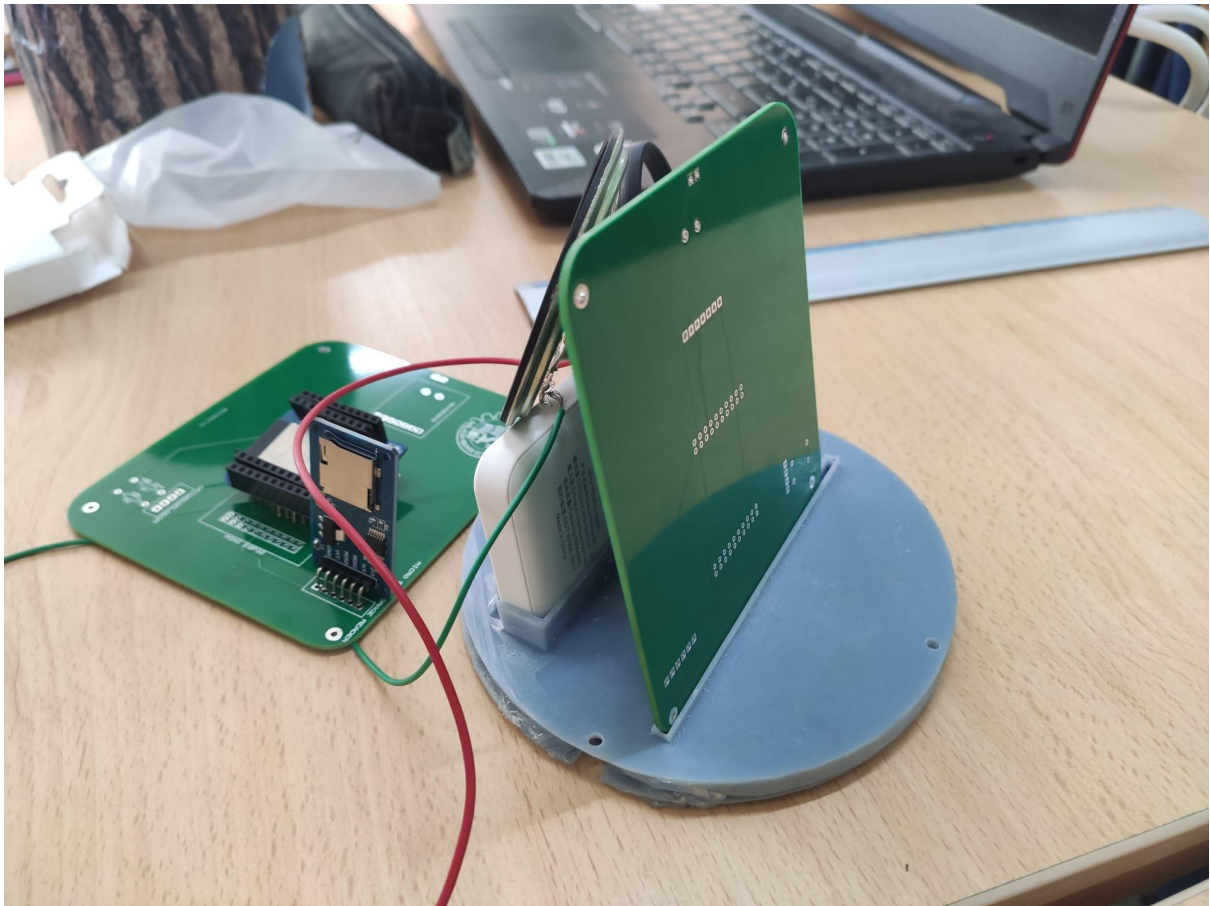
Parte estética:

Hemos usado cola blanca y papel impreso para la decoración exterior en forma de madera.







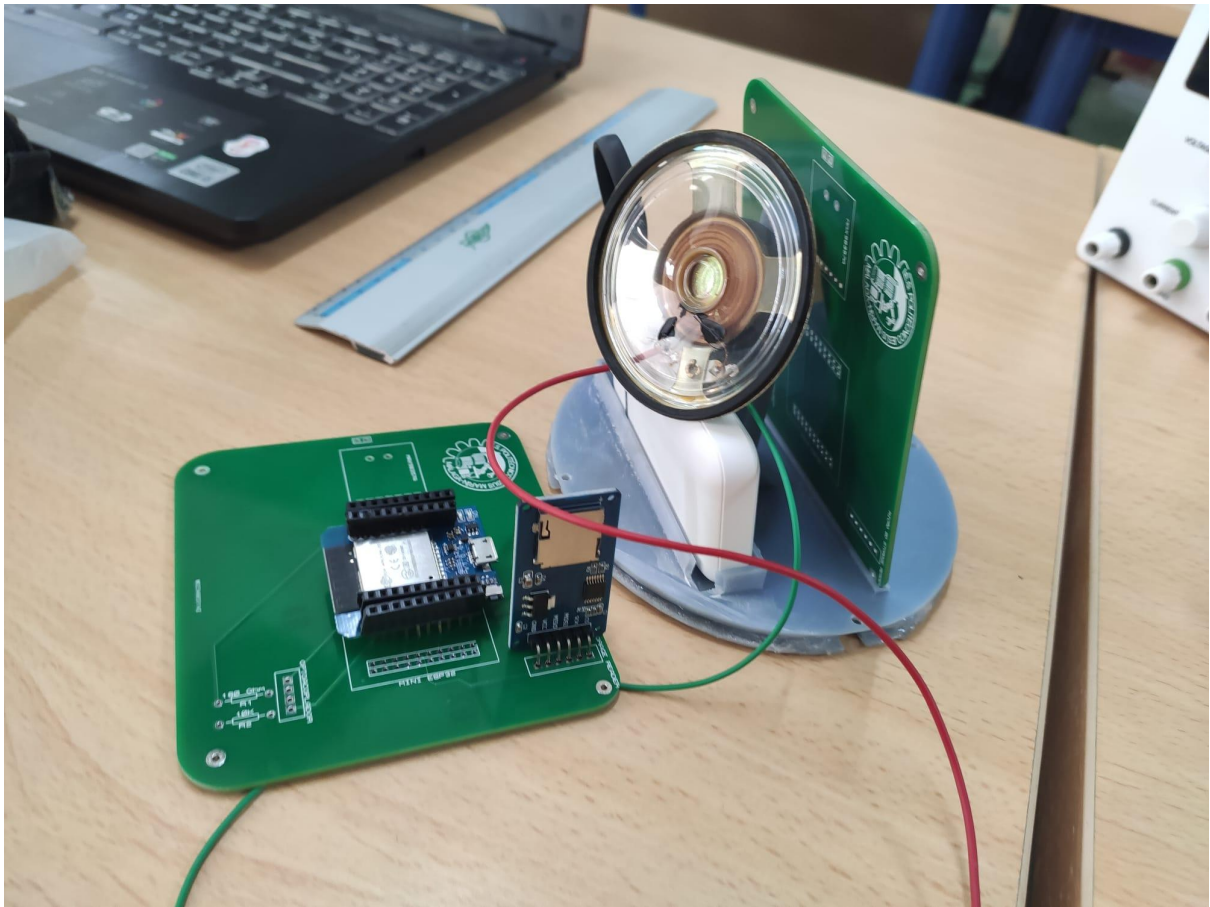


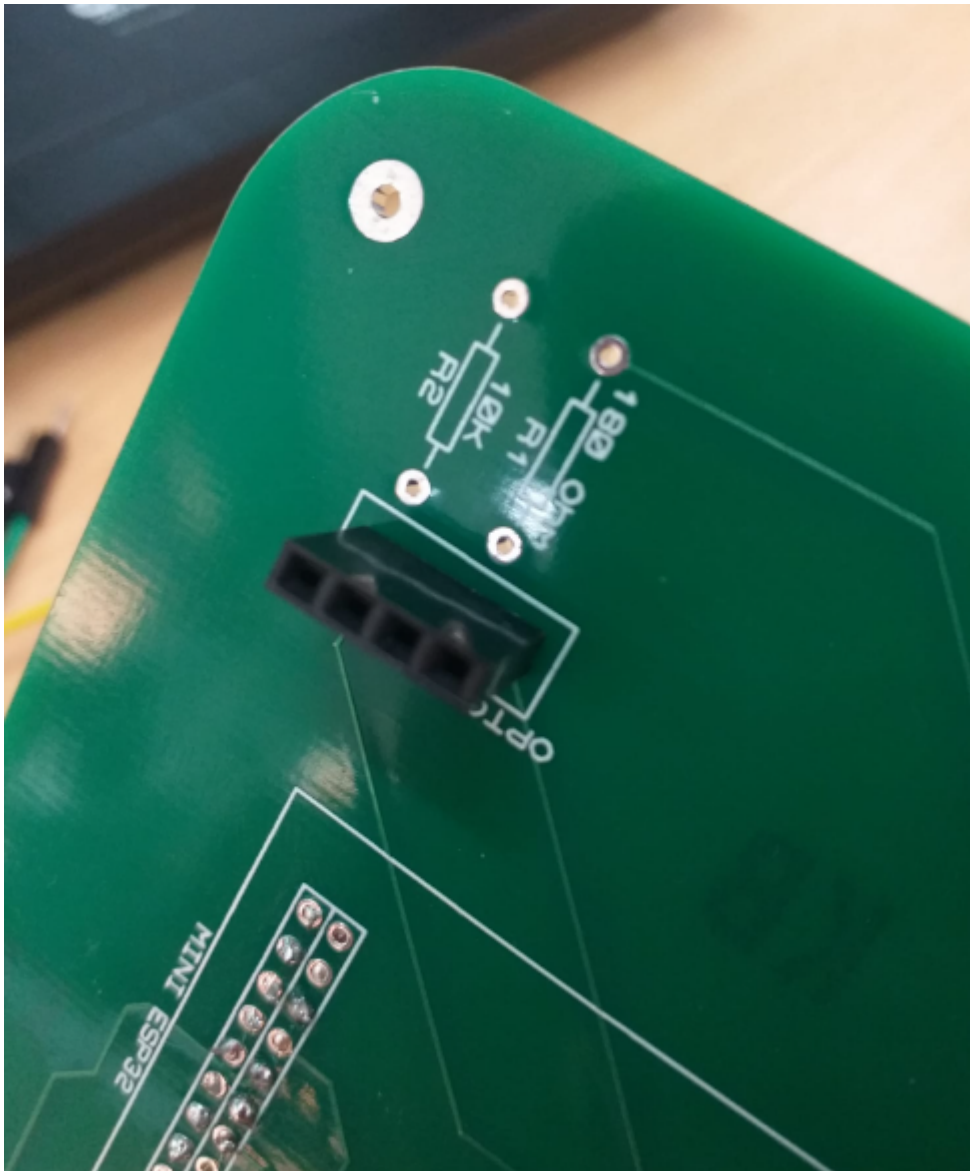
Aquí vemos el resultado de la impresión 3D. Todas las partes cuadran muy bien en su sitio. Ahora nos vamos a “pelear” con el conexionado, porque la mini esp32 parece que no encaja muy bien en su sitio al ser los pads demasiado alejados el uno del otro. Como medida casi rozando la desesperación, vamos a soldar en una mini Esp32 directamente el cableado sin los pines rígidos, ya que al estar las medidas erróneas, como observamos en la última imagen, tenemos que improvisar con los recursos que nos quedan.



Otro detalle que vemos a continuación es que la tarjeta SD va levantada. Le hemos soldado

unos pines para optimizar el espacio, y se pueda plegar paralelamente a la placa.



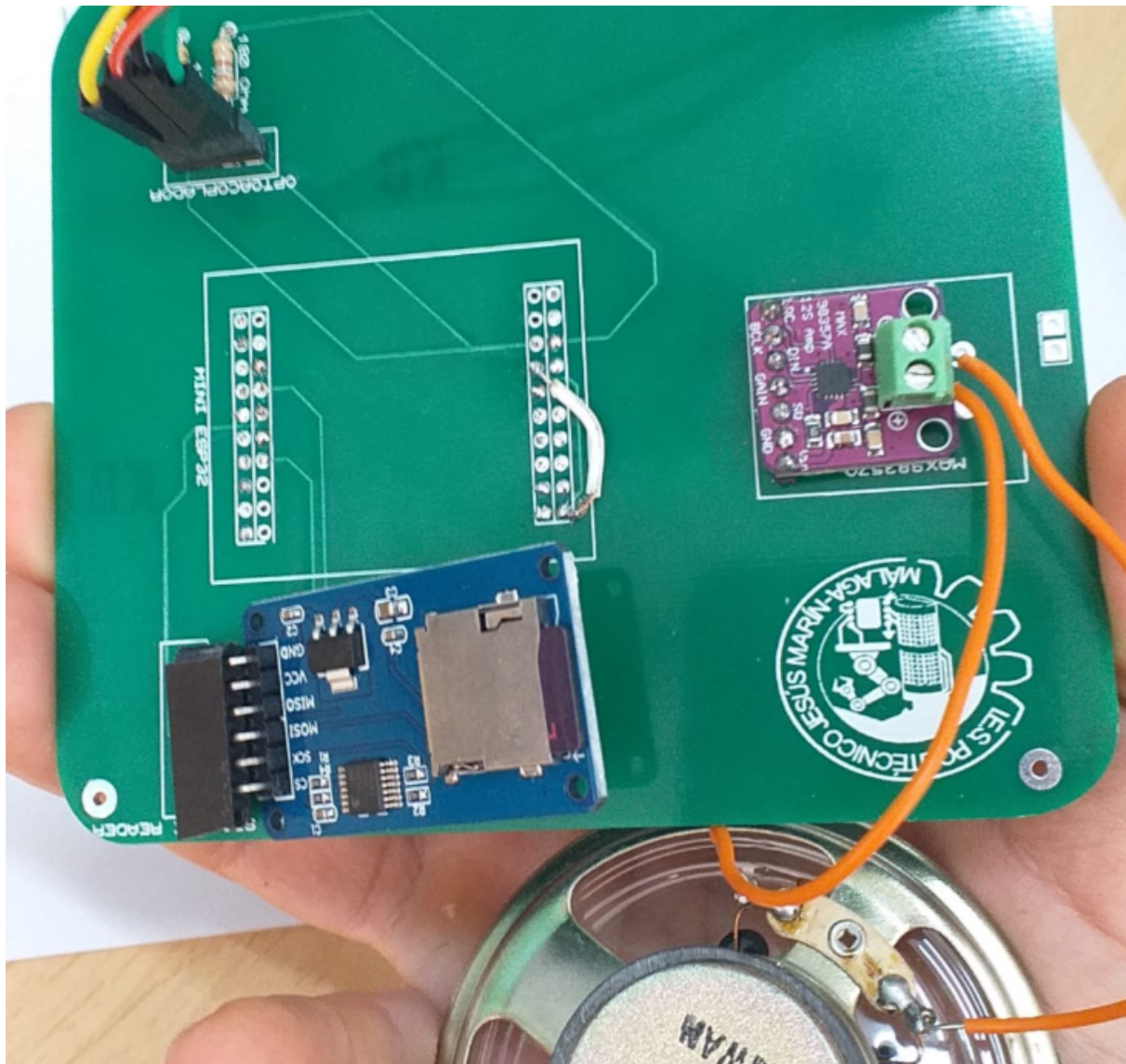


Errores importantes que hemos encontrado: si recordamos al principio de todo cuando tuve que crear los componentes porque no existían en la librería, nos daremos cuenta que persiste ese error que hemos arrastrado desde el principio, que todavía no sabemos por qué, las pistas se veían “gris” transparente en vez de azul oscuro. He creado las pistas y los componentes de nuevo y ha persistido el error. Ahora se traduce eso en que, al pasarse esto a Gerber, hay pistas que aleatoriamente se han perdido y no están.

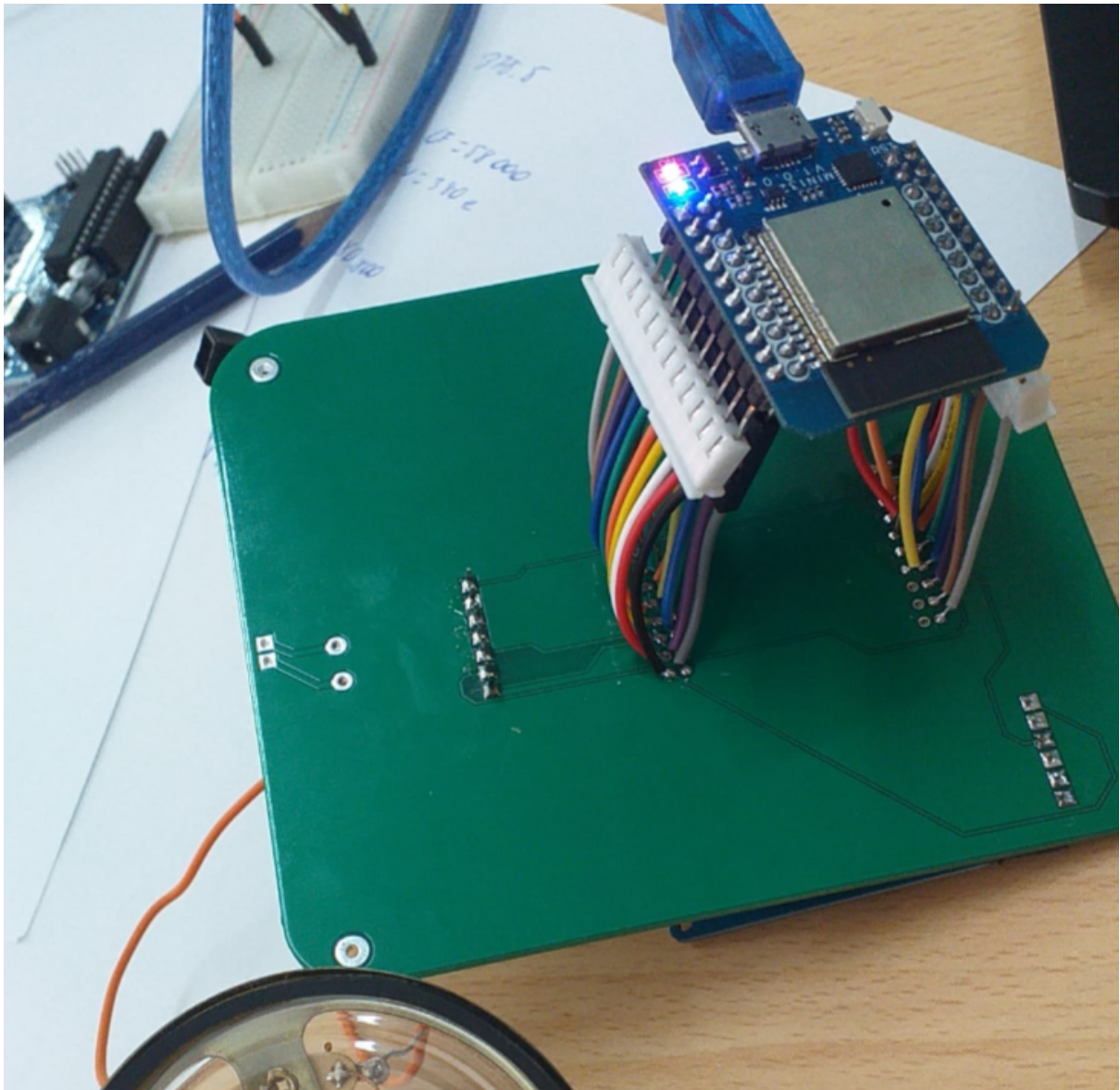
Hemos intentado corregir torpemente ese error soldando cables en aquellos sitios donde debería haber pistas. Esto evidentemente dificulta mucho que en la etapa final funcione el conjunto.



El ejemplo más claro está en las resistencias de 180 ohmios y de 10k.



También estamos teniendo problemas con la programación. No ha cambiado nada el código, pero la aplicación de arduino no reconoce la librería "Audio.h". Mañana en la última hora de clase intentaremos arreglar este bug.



Otro error importante es el tamaño de la placa mini Esp32. A pesar de haber medido una y otra vez para asegurarme, en algún momento debo haber movido algo y se ha descolocado todo, de modo que la placa quedó mucho más grande que la impresión. El “apaño” que se hizo para adaptar los tamaños fue a través de cableado. Esto posiblemente dificulte el encaje final en la maqueta, porque están variando los tamaños.